

Conversational SHL Assessment Recommender

Approach Document · Harsh Kadiyan

1. What Was Built

Instead of a keyword search box, this system gives recruiters a conversational agent that understands role descriptions in plain English and maps them to the right SHL assessments. The agent handles vague inputs, asks targeted follow-up questions, refines results when requirements shift, compares assessments side-by-side, and firmly rejects off-topic or adversarial requests — all in a multi-turn chat. The API is fully stateless as required by the assignment. Each `/chat` request contains the complete conversation history.

2. Architecture at a Glance

The system follows a Retrieval-Augmented Generation (RAG) pipeline:

- User conversation is analyzed
- Relevant SHL assessments are retrieved semantically
- Retrieved catalog data is injected into the LLM prompt
- The LLM generates grounded conversational responses

API Framework	FastAPI — lightweight, async-ready, clean routing
Embeddings	Sentence Transformers — semantic role/skill understanding
Vector Search	FAISS — fast top-k similarity retrieval at scale
Data Source	BeautifulSoup — SHL catalog scraping & normalization
LLM	Grok via OpenRouter — grounded conversational generation

3. Data Collection & Retrieval

The SHL Individual Test Solutions catalog was scraped with BeautifulSoup and cleaned into a structured dataset. Semantic search was chosen over keyword matching because recruiters naturally describe roles conversationally (e.g. "someone who can lead a team under pressure") — exact terminology matching would miss most of these queries.

For each assessment, the system stores:

- Assessment name
- SHL catalog URL
- Test type/category
- Description
- Skills and competencies

4. Conversation Flow

Four interaction patterns are handled cleanly:

- **Clarification** — vague requests (e.g. "I need an assessment") trigger targeted follow-up questions before any recommendation is made.
- **Recommendation** — once enough context is gathered, the agent returns a shortlist with name, URL, and test type.
- **Refinement** — mid-conversation changes (e.g. adding a personality test requirement) update the shortlist in-place rather than restarting.
- **Comparison** — catalog data for both assessments is retrieved and used directly, eliminating model hallucinations.

All responses are stateless — the full conversation history is passed in every /chat request, matching the assignment specification.

5. Prompt Engineering

Prompt design was the most iterative part of the project. Key constraints enforced in every prompt:

- URLs must come exclusively from the scraped catalog — no model-generated links.
- Recommendations are only made once sufficient role context exists.
- Off-topic requests (legal advice, general HR guidance, prompt injections) are refused politely but firmly.
- Response schema is strictly enforced so the API consumer receives predictable JSON every time.

6. Testing & Key Improvements

Manual testing covered five conversation patterns: *vague requests*, *mid-conversation refinements*, *comparison queries*, *off-topic questions*, and *prompt injection attempts*. Several iterations produced meaningful improvements:

- Better catalog cleaning reduced noisy retrieval results significantly.
- Stronger prompt constraints stopped early-turn over-recommendation.
- Improved ranking surfaced more relevant assessments for ambiguous role descriptions.
- Response size and latency were trimmed to stay within assignment constraints.

7. AI Tools Used

AI-assisted tooling (primarily LLM chat) was used for debugging, architecture brainstorming, and prompt refinement. All retrieval logic, integration decisions, and evaluation were manually implemented and validated.

~ Harsh Kadiyan